UNIDAD 5
TAREA 1

Tarea: Sistema de Gestión de Vehículos

Descripción:
Desarrollo un pequeño **sistema de gestión de vehículos** usando **Programación Orientada a Objetos (POO)** y aplicando la **estructura de código en Python** mediante **módulos y paquetes**.

Requisitos del ejercicio

1. **Crea un paquete llamado gestion_vehiculos** que contendrá:
   - Un módulo vehiculo.py con una clase base Vehiculo.
   - Un módulo coche.py con una clase Coche que herede de Vehiculo.
   - Un módulo moto.py con una clase Moto que herede de Vehiculo.

2. **Clase Vehiculo (en vehiculo.py)**
   - Atributos:
     - marca (cadena de texto)
     - modelo (cadena de texto)
     - anio (entero)
   - Método descripcion(): Devuelve una cadena con la información del vehículo.
   - Método arrancar(): Imprime "El vehículo está arrancando...".

3. **Clase Coche (en coche.py)**
   - Hereda de Vehiculo.
   - Añade un atributo num_puertas (entero).
   - Sobrescribe descripcion() para incluir el número de puertas.

4. **Clase Moto (en moto.py)**
   - Hereda de Vehiculo.
   - Añade un atributo tipo (cadena, valores posibles: "deportiva", "scooter", "naked").
   - Sobrescribe descripcion() para incluir el tipo de moto.

5. **Archivo main.py** (en la raíz del proyecto)
   - Importa las clases desde el paquete gestion_vehiculos.
   - Crea una instancia de Coche y otra de Moto, mostrando sus descripciones y llamando a arrancar().

La estructura del proyecto debe ser la que sigue:

```
proyecto/
├── gestion_vehiculos/
│   ├── __init__.py
│   ├── vehiculo.py
│   ├── coche.py
│   └── moto.py
└── main.py
```

Entrega:

Entrega tu trabajo en un **fichero comprimido** con todos los archivos necesarios.

Criterios de calificación:

El ejercicio tiene una puntuación total de 10 pts.

Los criterios de calificación son los siguientes:

TAREA5_1		Se ajusta a lo que se pide	Realiza lo que se pide con errores	Realiza sólo la mitad de lo que se pide	Sólo realiza parte de lo que se pide	No se ajusta a lo que se pide
		9/10	7/8	5/6	3/4	1/2
Corrección del Código	El código funciona sin errores y cumple los requisitos del ejercicio (clases, herencia, métodos e importaciones).50%					
Organización y Claridad	El código está bien estructurado en módulos, sigue buenas prácticas de programación (indentación, nombres adecuados, comentarios básicos). 30%					
Funcionamiento del Programa	Se ejecuta correctamente y muestra la salida esperada en main.py. 20%					
